NOVA IMS INFORMATION MANAGEMENT SCHOOL

From strings to lists

Computation I · Lecture 04

Karina Brotto Rebuli
krebuli@novaims.unl.pt

J.C.R. Licklider

1960

The problem: computers used to take hours to answer. You handed a deck of punched cards to an operator, walked away, and came back later for the printout. Licklider wanted to think with the machine — question, answer, question — in the time thinking takes.



Image source

Three questions today

How a program asks.

What a list can do.

How a string becomes a list.

01.

The program asks

Talking at run time

The line every deck ends with

Run by hand last week — watch it again

```
input("Questions? ")
```

```
Questions?
```

LIVE

The program stops and waits

Be clear about what the computer is waiting for

```
x = input()
print(x)
```

```
lisbon
lisbon
```

LIVE

Say what you are waiting for

The argument is the prompt

```
name = input("Your name: ")
print("Hello, " + name + "!")
```

```
Your name: World
Hello, World!
```

input() always returns a string

Even when you type digits

```
x = input("Write a number: ")
print(type(x))
```

```
Write a number: 7
<class 'str'>
```

Ask for a number, square it

Predict before you run

```
x = input("Write a number: ")
print(x ** 2)
```

```
Write a number: 5
TypeError: unsupported operand type(s) for ** or pow(): 'str' and 'int'
```

LIVE

Convert before you calculate

The converters you already own

```
x = int(input("Write a number: "))
print(x ** 2)
```

```
Write a number: 5
25
```

Concatenating a number with text

Predict — it looks innocent

```
x = int(input("Write a number: "))
print("The result is " +  x ** 2)
```

```
Write a number: 2
TypeError: can only concatenate str (not "int") to str
```

Two ways to dress a number to be printed

str() converts, then + glues it; an f-string does the converting for you

```
x = int(input("Write a number: "))
print("The result is " + str(x ** 2))
print(f"The result is {x ** 2}")
```

```
Write a number: 2
The result is 4
The result is 4
```

LIVE

`print` takes more than one thing

Several values, a separator, and what comes at the end

```
print("Year:", 2026)
print("21", "09", "2026", sep="/")
print("Year:", end=" ")
print(2026)
```

```
>>> Year: 2026
>>> 21/09/2026
>>> Year: 2026
```

The Console

Python, one line at a time

```
>>> 5 + 9
14
>>> help(print)
>>> help(input)
```

What does this print?

```
x = input("Number: ")
y = float(x) * 2
print(x + str(y), y)
print(f"{x}{y}", x * 2)
```

```
Number: 2.5
2.55.0 5.0
2.55.0 2.52.5
```

A list was born

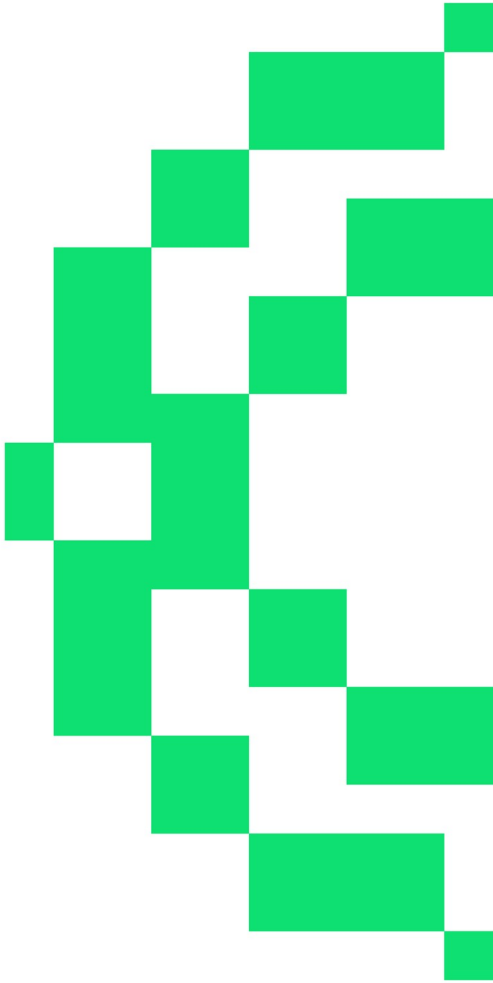
Exercise 5, replayed

```
date = "21/09/2026"
print(date.split("/"))
```

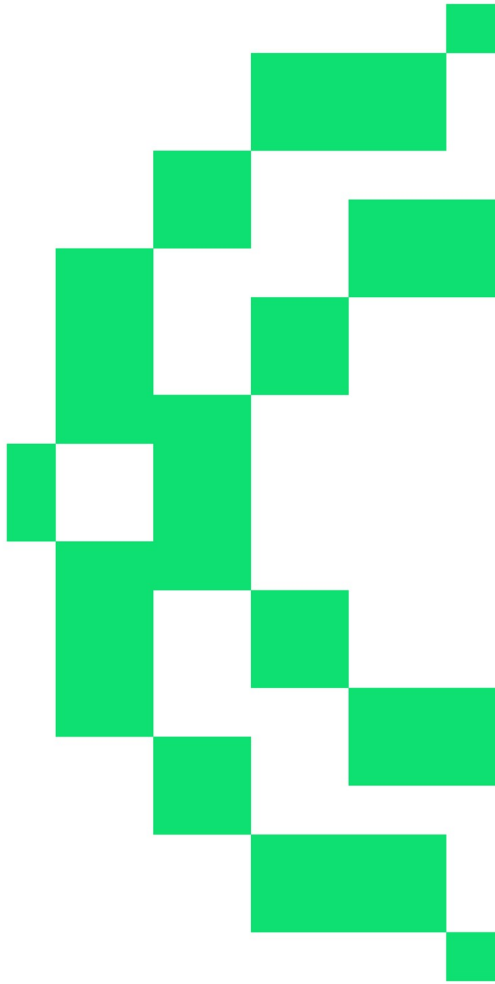
```
>>> ['21', '09', '2026']
```

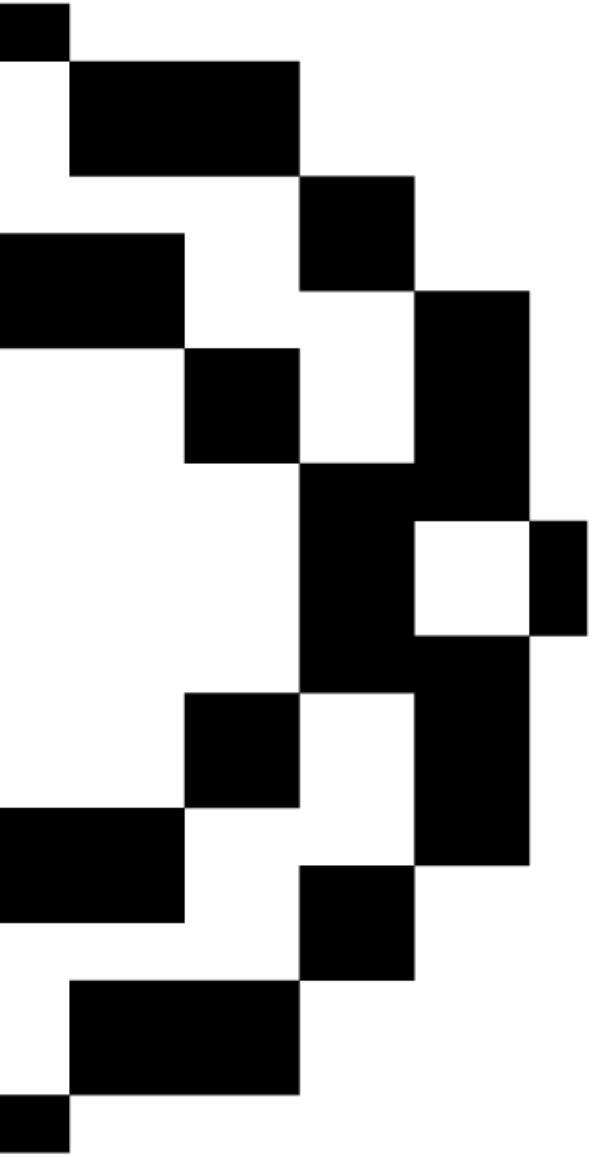
It created an object with a new type: a list.

A container born from a string.



```
"""The list holds any types, and it can be
modified."""
```





02.

The sequence that changes

Lists, formalized

A list, formalized

Any values, in order

```
L = [31, "march", True]
print(L)
print(len(L))
```

```
>>> [31, 'march', True]
>>> 3
```

The grammar you already know

Index it like a string

```
L = [31, "march", True]
print(L[0])
print(L[-1])
```

```
>>> 31
>>> True
```

The line strings refuse, but lists accept

With a list, you can change it in place

```
L = [10, 12, 17, 9]
print(L)
L[1] = 15
print(L)
```

```
>>> [10, 12, 17, 9]
>>> [10, 15, 17, 9]
```

LIVE

The slice can take a third number

start : stop : step — on lists and on strings alike

```
L = [0, 1, 2, 3, 4, 5]
print(L[1:4])
print(L[::2])
print(L[::-1])
```

```
>>> [1, 2, 3]
>>> [0, 2, 4]
>>> [5, 4, 3, 2, 1, 0]
```

LIVE

Is it in the list?

It checks if the value is an item of the sequence

```
grades = [10, 12, 17]
print(12 in grades)
print(9 in grades)

L = [31, "march", True]
print("m" in  L)
```

```
>>> True
>>> False
>>> False
```

Three ways to add elements to a list — append, extend, and +

The first two change the list itself; + builds a new list and leaves this one alone

`append`

WHAT IT DOES

Puts the argument at the end of the list as one single element.

THE ORIGINAL LIST

Changes in place

USE IT WHEN

one more value arrives.

`extend`

WHAT IT DOES

Runs through the argument and adds each of its elements.

THE ORIGINAL LIST

Changes in place

USE IT WHEN

a whole collection of values arrives.

`+`

WHAT IT DOES

Joins two lists together and hands back the result.

THE ORIGINAL LIST

Stays as it was

USE IT WHEN

the original must be kept untouched.

LIVE

Three ways to add elements, in practice

```
grades = [10, 12]
print(grades)
grades.append(17)
print(grades)
grades.extend([20])
print(grades)
grades + [9]
print(grades)
```

```
>>> [10, 12]
>>> [10, 12, 17]
>>> [10, 12, 17, 20]
>>> [10, 12, 17, 20]
```

LIVE

+ joins a list with a list, only

One of these lines refuses

```
print([1, 2] + [3])
print([1, 2] + 3)
```

```
>>> [1, 2, 3]
TypeError: can only concatenate list (not "int") to list
```

The same star, on a list

Operator overloading again: * repeats the sequence

```
print("ab" * 3)
print([0] * 10)
print([1, 2] * 3)
```

```
>>> ababab
>>> [0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
>>> [1, 2, 1, 2, 1, 2]
```

Three ways to take elements out of a list — remove, pop, and del

Two methods and a statement — all three change the list itself; they differ in how you point at the element

`remove`

WHAT IT DOES

Searches for the value and takes out the first match.

CHOOSES BY

The value

USE IT WHEN

you know the value, not where.

`pop`

WHAT IT DOES

Takes out the element at a position — with no argument, the last one.

CHOOSES BY

The position

USE IT WHEN

you know the position.

`del`

WHAT IT DOES

Deletes the element at a position — a statement, not a method call.

CHOOSES BY

The position, or a whole slice

USE IT WHEN

you know the position — one or a slice.

LIVE

Three ways to take elements out

remove by value, pop by position, del by position

```
grades = [10, 12, 17, 9]
grades.remove(12)
grades.pop()
del grades[0]
print(grades)
```

```
>>> [17]
```

Where is this one broken?

It runs. Nobody complains.

```
grades = [10, 12]
grades = grades.append(17)
print(grades)
```

```
>>> None
```

03.

From string to list

Cutting at the separator

Cut at the separator

The debt from exercise 5, paid

```
date = "21/09/2026"
print(date.split("/"))
```

```
>>> ['21', '09', '2026']
```

> **<obj>.split(sep, maxsplit)**
>
> **sep**
> — *optional*. The separator. Default: any whitespace.
>
> **maxsplit**
> — *optional*. Cuts allowed, at most. Default -1: every one.

list() cuts at *every* character

A converter, like int() and float() — blunt on purpose

```
print(list("2026"))
print(list("21/09/2026"))
```

```
>>> ['2', '0', '2', '6']
>>> ['2', '1', '/', '0', '9', '/', '2', '0', '2', '6']
```

Three weeks in three lines

Receive a date from the user and print the next year — use input, split, index

```
text = input("Day/month/year: ")
parts = text.split("/")
print(f"Next year: {int(parts[2]) + 1}")
```

```
Day/month/year: 21/09/2026
Next year: 2027
```

Three questions, answered

In one sentence each

A program asks
with input — and the answer arrives as a string, always

A list
holds any types and can be modified: index it, change an element, concatenate it, append to it

A string becomes a list
with split — cut at the separator, every piece still a string

Four weeks, one map

Everything so far, in one line each

WEEK 1

What a program is, and raised vs. silent errors

A text file the interpreter runs, top to bottom — wrong in three ways: syntax, runtime, semantic

WEEK 2

Variables anatomy and data types in Python

int, float, bool, str — variables name them, converters change them, operators calculate and compare

WEEK 3

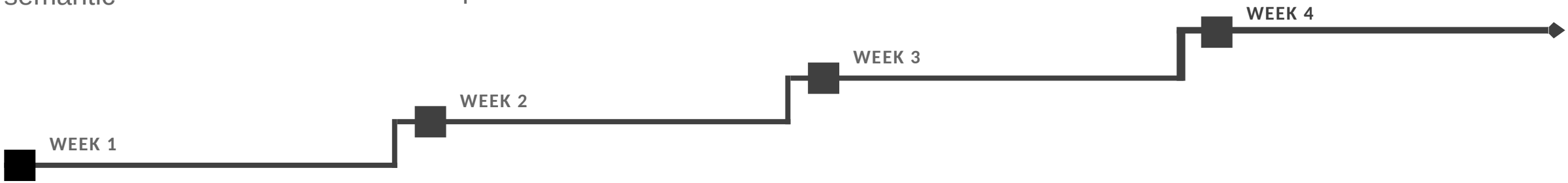
The string, the immutable sequence

index it, slice it, glue it, repeat it, ask with in, call its methods — but it never changes

WEEK 4

The program asks, and the list as the mutable sequence

input answers with a string; the list holds any types and can be modified; split bridges the two



J.C.R. Licklider

1960

Every input you run stops, waits, and answers in the time thinking takes.

Nobody walked away for the printout.

Before the next theoretical lecture

On your own

Think Python — Sections 5.11 Keyboard input · 9.1 A list is a sequence · 9.2 Lists are mutable · 9.3 List slices · 9.4 List operations · 9.5 List methods · 9.6 Lists and strings · 9.13 Debugging.

One habit — if you get lost with the list values, print the list after every line that changes it.

Exercise 1 · Predict the value

Read the program below and write exactly what it prints. On paper, before any run. Then run it and compare.

```
L = [3, 5]
L.append(7)
print(L[-1])
```

Exercise 2 · Predict what it prints

One of these lines refuses to run. On paper: write what the first print shows, then say which line stops the program, and why. Then run it and compare.

```
L = [10, 12]
L.append(17)
print(12 in L)
print(L + 17)
```

Exercise 3 · The 8 should be an 18 — fix the grade

Write the line that fixes the grade: the 8 should be an 18. Start from the list given in the panel and change it in place — this lecture's mutation by index, the exact move strings refused last week. Then print the list to prove it.

```
grades = [10, 8, 17]
# your lines here
```

Exercise 4 · Answer in capitals

Finish the program — one line is missing. The input line is already written in the panel; it hands you whatever the user typed. Print that answer in capitals: last week's string methods, on this week's input.

```
city = input("City: ")
# your line here
```

Exercise 5 · `in` on a string, `in` on a list

Read the program below and write exactly what it prints — three lines. On paper, before any run. Then run it and compare. It is the same in, asked of a string and then of a list.

```
word = "lisbon"
parts = ["lis", "bon"]
print("bon" in word, "bon" in parts)
print("o" in word, "o" in parts)
print(len(word), len(parts))
```

Exercise 6 · Extend with a string

Read the program below and write exactly what it prints — the list, then its length. On paper, before any run. Then run it and compare.

```
grades = [10, 12]
grades.extend("17")
print(grades)
print(len(grades))
```

Exercise 7 · Split, then index

Read the program below and write exactly what it prints — three lines. On paper, before any run. Then run it and compare. Everything in it is this lecture's split and indexing, with week 3's slice on the last line.

```
line = "12 7 2026"
parts = line.split()
print(parts[0] + parts[1])
print(int(parts[0]) + int(parts[1]))
print(parts[-1][:2])
```

Exercise 8 · One more grade, at the end

The program should print [10, 12, 17]. It stops before the print instead. Find the line to blame and write the fix — there are two, and this lecture's table of append, extend and + has both. Then run it and compare.

```
grades = [10, 12]
grades.extend(17)
print(grades)
```

Exercise 9 · Two grades, one list

Write a program that asks for two grades, one at a time, and prints them as a list of numbers, then their average. Use this lecture's input and int, and a list you write with brackets; week 2's arithmetic does the rest. One run of the finished program, for example:

```
Grade: 10
Grade: 12
[10, 12]
11.0
```

NOVA IMS INFORMATION MANAGEMENT SCHOOL

```
input("Questions? ")
```

Acreditações e Certificações da NOVA IMS

eduniversal · QS STARS · ABET Computing Accreditation Commission · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · Lapcer · IQNet

Cofinanciado por

PORTUGAL 2030 · 2030 Lisboa Programa Regional de Lisboa · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU

© Karina Rebuli, 2026

Exercise 10 · Ask for an age, then answer twenty ★

Read the program below, then run it twice — once answering 20, once answering twenty.

On paper first: what do you expect each run to do? Compare, and describe the difference in your own words.

How would you fix it? Answer in plain words, without worrying about Python syntax or implementation.

```
age = int(input("Age: "))
print(age + 1)
```